

Comment optimiser l'exécution d'une requête SQL à l'aide de l'algèbre relationnelle ?

Eliott PAQUET N° 30960

2025 – 2026

Les bases du traitement des requêtes SQL

Les bases du traitement des requêtes SQL

Le traitement d'une requête

Notations

l'algèbre relationnelle

Les optimisations naïves

Optimisation dépendante des données

Analyse des résultats

Conclusion

Ressource

Description du traitement d'une requête

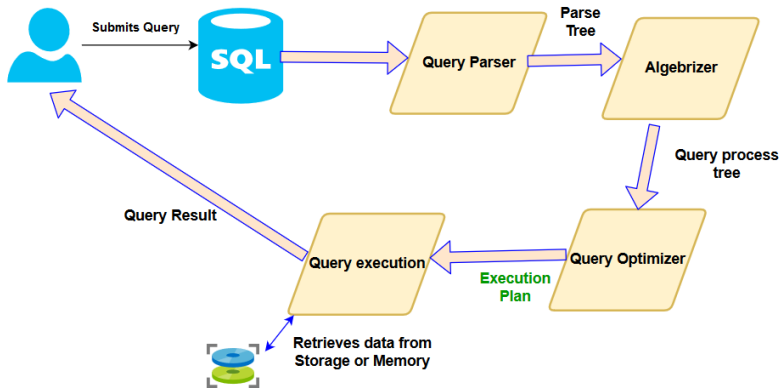


Figure: Traitement d'une requête SQL par un SGBD.

Source: Quest

Rappel et Notation

- ▶ On utiliseras les symboles: Π pour la projection, σ pour la selection et $\bowtie$ pour la jointure entre deux tables
- ▶ A chaque étape de l'arbre, n fera référence au nombre de ligne encore présente dans la table
- ▶ Les arbres d'exécution se lisent du bas vers le haut, la dernière projection représentant la toute dernière opération effectué
- ▶ Le jeux de données utilisé:
 - 3 Tables
 - 100000 lignes par table
 - 10000 requête lancé par paramètre
 - 100 requêtes différentes et très variées
 - 1 point vaut une moyenne de 100 requête

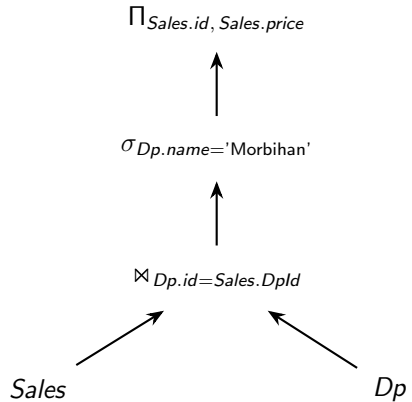
Edgar F. Codd (1970)

Calcul relationnel $\iff$ Algèbre relationnelle

```

1 SELECT S.id, S.price
2 FROM Sales
3 JOIN Dp ON Dp.id = S.DpId
4 WHERE Dp.name = 'Morbihan';

```



Les optimisations naïves

Les bases du traitement des requêtes SQL

Les optimisations naïves

- La descente des sélections

- Insertion de projections

- Méthodes de jointure

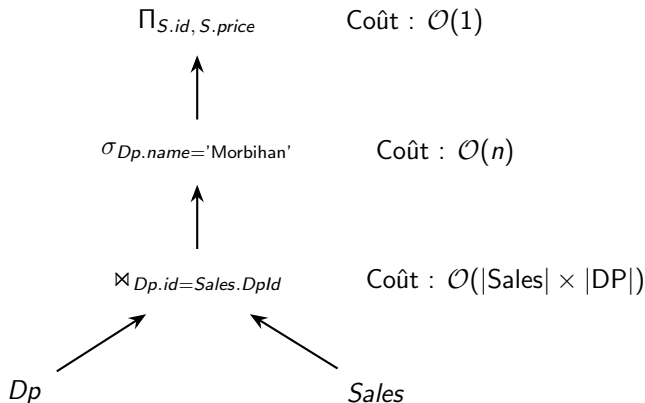
Optimisation dépendante des données

Analyse des résultats

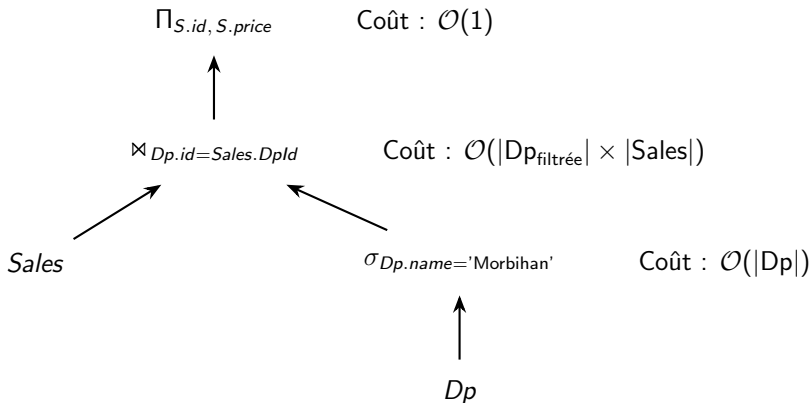
Conclusion

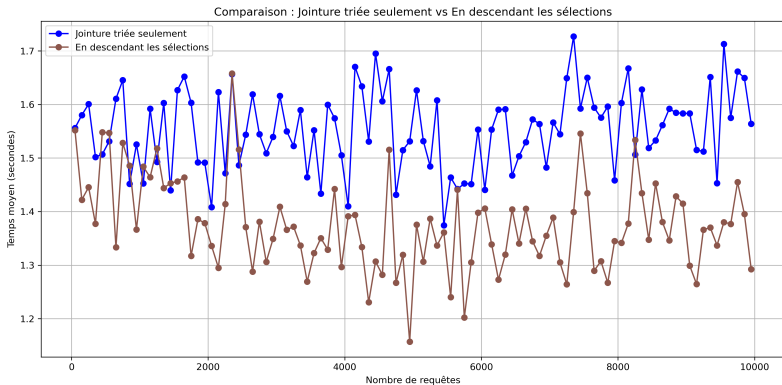
Ressource

La descente des sélections

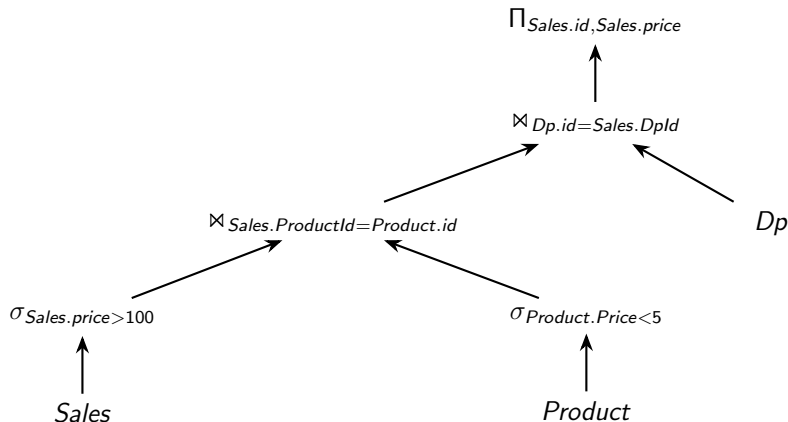


La descente des sélections

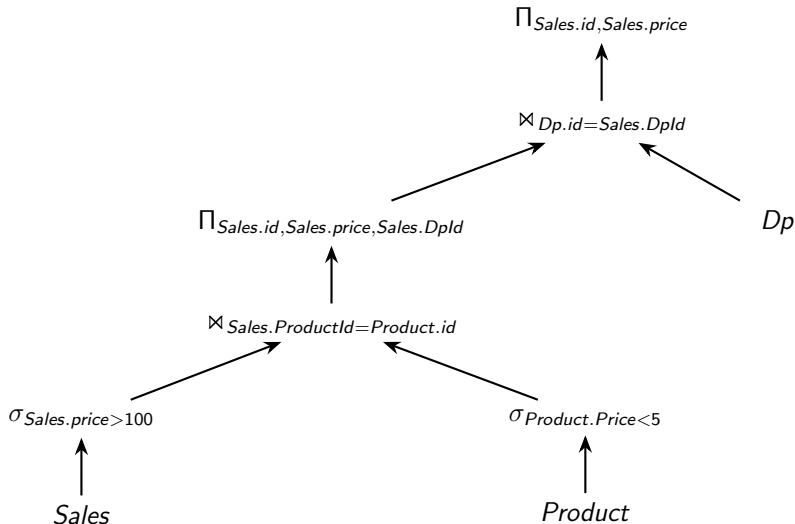


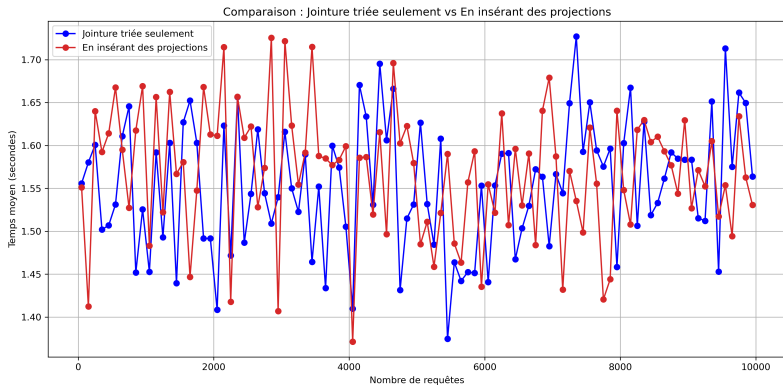


Cas problématique



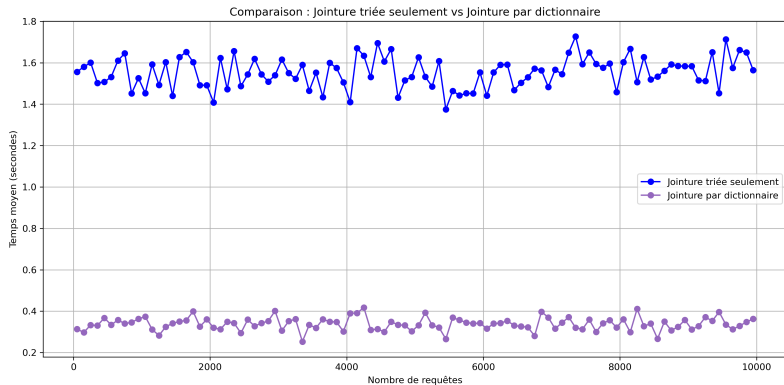
Insertion utile de la Projection





Les différents types de jointures possibles

- ▶ Produit Cartésien: génère tous les couples possibles et teste la condition de jointure $\rightarrow \mathcal{O}(n^2)$
- ▶ Jointure Triée: pré trie les deux tables et lit les deux tables $\rightarrow \mathcal{O}(2n \log(n) + 2n)$
- ▶ Jointure par dictionnaire: crée un dictionnaire et recherche les correspondances $\rightarrow \mathcal{O}(2n)$



Optimisation dépendante des données

Les bases du traitement des requêtes SQL

Les optimisations naïves

Optimisation dépendante des données

- Optimisation des expression binaire

- Choix de l'ordre des jointures

- Comment choisir le meilleur plan rapidement?

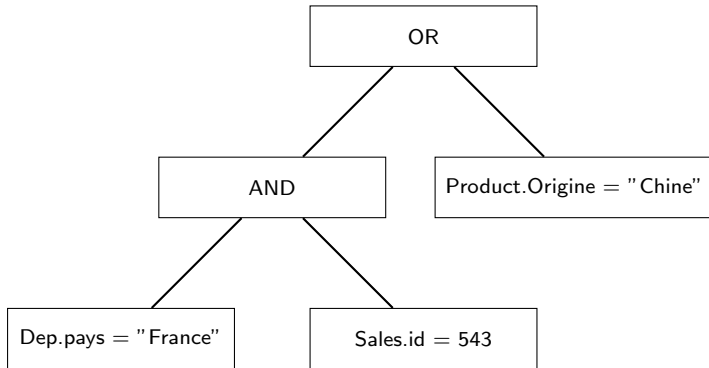
Analyse des résultats

Conclusion

Ressource

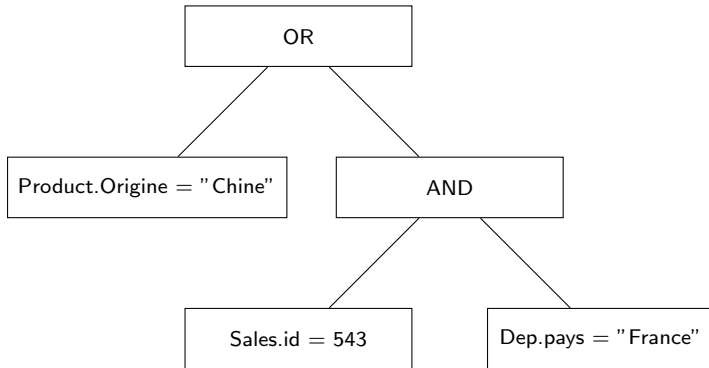
Analyse des expressions binaires

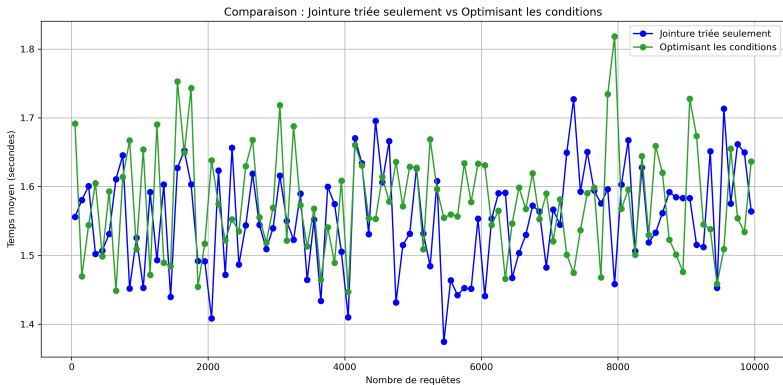
```
1 Where (Dep.pays = "France" AND Sales.id = 543) OR (Product.  
    Origine = "Chine")
```



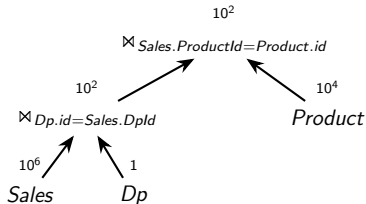
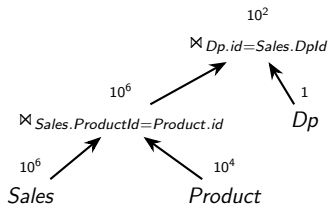
Optimisation des expressions binaires

```
1 Where (Dep.pays = "France" AND Sales.id = 543) OR (Product.  
    Origine = "Chine")
```

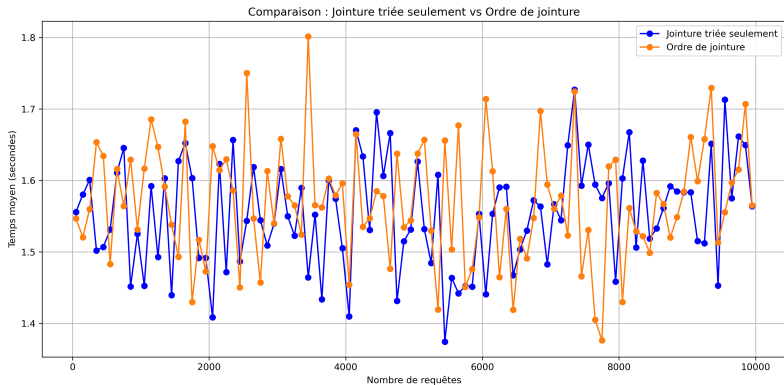




Résultats équivalents mais complexité différente



- ▶ $\bowtie_{Sales.ProductId=Product.id}$ d'abord $\rightarrow 10^4 \times 10^6 + 10^6 \times 1$ comparaisons
- ▶ $\bowtie_{Dp.id=Sales.Dpld}$ d'abord $\rightarrow 1 \times 10^6 + 10^2 \times 10^4$ comparaisons



Comment choisir le meilleur plan rapidement?

- ▶ Choisir les optimisations avant de lancer la requête
- ▶ Utiliser l'historique des plans générés
- ▶ Donner un coût à chaque opération, énumérer tous les plans et prendre celui avec le plus faible coût

Analyse des résultats

Les bases du traitement des requêtes SQL

Les optimisations naïves

Optimisation dépendante des données

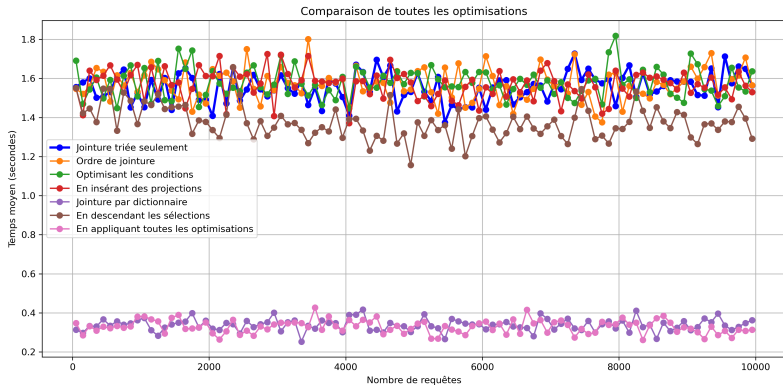
Analyse des résultats

Grand jeux de donnée

Critique

Conclusion

Ressource



Les raisons de la lenteur de notre modèle

- ▶ De nombreuses techniques n'ont pas été traitées, notamment les plus récentes (~ 20 ans)
- ▶ Les failles de notre code. 45 minutes $\rightarrow$ 22 secondes
- ▶ Les optimisations matérielles: Appel aux fonctions internes au processeur, le multi-threading
- ▶ La compression des données. Exemple: *C-Store* 4 manière de compresser les données
- ▶ L'indexation des données facilitent les jointures

Conclusion

Les bases du traitement des requêtes SQL

Les optimisations naïves

Optimisation dépendante des données

Analyse des résultats

Conclusion

Conclusion

Ouverture

Nos ressources

Ressource

Résumé

- ▶ Différentes optimisations naïve utilisant l'algèbre relationnelle: Descente de Selection et insertion de projection
- ▶ Certaines optimisations dépendantes des données que notre SGBD possède

Ouverture

Les pistes récentes:

- ▶ Analyse des données $\Rightarrow$ Machine Learning (Génération de plan, Choix de l'ordre des jointures,...)
- ▶ Une meilleure organisation des données en mémoire (Voire le travail d'Antoine)
- ▶ Les deux: *Hypothetical Index Benefit Estimation on Column-oriented Databases using Quantiles 2025*

Nos ressources

Nos ressources:

- ▶ *The Design and Implementation of Modern Column-Oriented Database Systems* 2013
- ▶ *C-Store: A Column-oriented DBMS* 2005
- ▶ *A Survey on Advancing the DBMS Query Optimizer: Cardinality Estimation, Cost Model, and Plan Enumeration* 2021
- ▶ *Extensible Query Optimizers in Practice* 2024
- ▶ *Foundation of Databases* 1995

Ressource

Les bases du traitement des requêtes SQL

Les optimisations naïves

Optimisation dépendante des données

Analyse des résultats

Conclusion

Ressource

La gestion mémoire

Notre gestion des tables pendant l'exécution

Les bases de données orientées colonnes

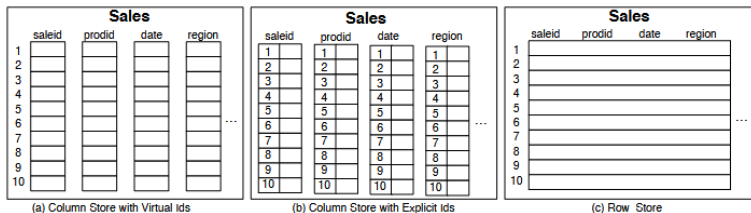
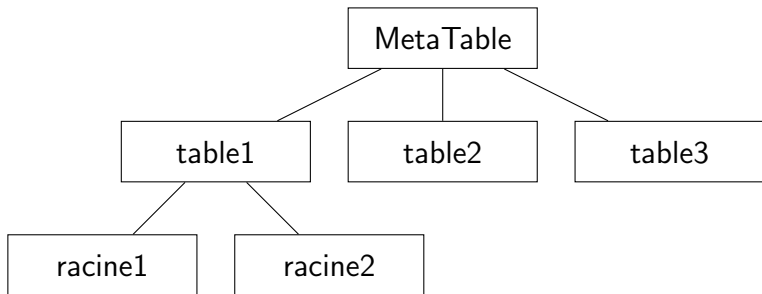


Figure: Rendu physique des BDD orientées colonnes et BDD orientées lignes

Notre gestion des tables pendant l'exécution



- **Racine**: valeurs chargées.
- **Table**: liste de lignes encore valides.
- **MetaTable**: jointure de plusieurs *Tables*.
- **Particularité**: modifier les lignes d'une *Table* (jointure ou sélection) modifie toutes les *Tables*.